

福州工商学院

FUZHOU TECHNOLOGY AND BUSINESS UNIVERSITY



设计说明书

课程名称:	单片机原理与接口技术
设计名称:	电子时钟设计
学 院:	新能源汽车与工程学院
年级专业:	2025 级电子信息工程（专升本）
学 号:	2550310199
姓 名:	叶嘉雄
任课教师:	范日高 成绩:

2026 年 5 月 23 日

电子时钟设计

摘 要

本文设计并实现了一款基于 STC89C52 单片机的电子时钟系统。系统采用 LCD1602 液晶显示屏实时显示年、月、日、时、分、秒及星期信息，支持 12/24 小时制切换；通过七路独立按键完成时间日期设置、闹钟设定及界面切换等交互操作；利用串口通信控制语音模块实现 8:00-22:00 整点报时功能；闹钟到达设定时间时驱动有源蜂鸣器报警。软件设计采用模块化编程，以定时器中断提供 1ms 系统时基，通过分层进位算法维护时间变量，并运用基姆拉尔森公式计算星期。在 Proteus 仿真环境下完成了系统调试与功能验证。实际运行结果表明，该统计时准确、操作简便、人机交互友好，具有较高的实用性与可扩展性，可作为嵌入式系统教学的实践案例。

关键词：STC89C52；电子时钟；LCD1602；闹钟；整点报时；串口通信；

目 录

第一章 前言.....	1
1.1 研究背景.....	1
1.2 设计任务与要求.....	1
第二章 系统整体概述.....	2
2.1 系统整体方案.....	2
第三章 系统硬件设计.....	2
3.1 系统主控芯片.....	2
3.2 LCD1602 液晶显示屏	3
3.3 七路独立按键.....	3
3.4 有源蜂鸣器	4
3.5 语音报时模块.....	5
3.6 Proteus 仿真设计	6
第四章 系统软件设计.....	7
4.1 软件整体框架	7
4.2 LCD1602 液晶屏显示数据	7
4.3 七路独立按键.....	8
4.4 有源蜂鸣器	9
4.5 语音报时模块.....	10
4.5.1 串口通信初始化.....	10
4.5.2 串口通信协议.....	11
第五章 核心算法.....	12
5.1 时间核心逻辑.....	12
5.1.1 时间核心逻辑简述.....	12
5.1.2 计算星期核心逻辑.....	13
第六章 调试与实现.....	16
6.1 软件调试与分析.....	16
6.2 proteus 仿真操作方法	16
6.2.1 主界面操作方法.....	17

6.2.2 时间设置操作方法.....	17
第七章 总结.....	20
第八章 参考文献.....	21
第九章 附录.....	22

第一章 前言

1.1 研究背景

随着嵌入式技术的飞速发展，单片机在智能仪表、消费电子及工业控制领域的应用日益广泛。电子时钟作为单片机应用最基础且典型的实例，不仅要求具备精准的计时功能，还需集成人性化的人机交互界面与辅助功能。

本课程设计基于 STC89C52 单片机，旨在开发一款电子时钟设计。利用 C 语言进行模块化编程，实现了包括日期显示、时钟计时、星期自动计算、闹钟设定及 12/24 小时制切换在内的核心功能。本设计不仅验证了单片机定时器中断、按键扫描及 LCD1602 显示驱动等关键技术，还重点探讨了时间进位逻辑与闰年判断算法的实现。

1.2 设计任务与要求

实现年、月、日、星期、时、分、秒的显示；能实现 12/24 小时制的切换，并能进行调时；实现 8:00-22:00 的整点报时功能；能够设置定时时间和修改定时时间，当到达定时时间时，发出报警，即实现闹钟功能。

第二章 系统整体概述

2.1 系统整体方案

本系统以宏晶科技（STC）的 **STC89C52** 作为主控芯片，结合 Proteus 仿真平台进行虚拟开发与验证。该芯片是一款高性能、低功耗的 8 位微控制器，内置 8K 字节在系统可编程 Flash 存储器，通过 MCS-51 单核处理器与外围电路协同工作，利用定时器中断实现精准时序控制。系统基于 Proteus 仿真环境搭建硬件电路模型，通过软件编程与虚拟调试，实现电子时钟的核心功能。系统主要组成部分及预设实现目标功能如下：

表 2-1 系统主要组成部分和及预设实现目标功能

系统组成	目标功能
STC89C52 主控芯片	协调各模块时序，处理按键输入、驱动显示与语音报时
LCD1602 液晶显示屏	显示“主菜单”“调时菜单”“闹钟菜单”
7 路独立按键	实现人机交互
有源蜂鸣器	支持自定义闹钟设定及开关控制
语音报时模块	串口驱动语音芯片，实现 整点自动报时
Proteus 仿真平台	构建虚拟硬件电路，模拟系统运行。

第三章 系统硬件设计

3.1 系统主控芯片

在系统主控芯片上，择选宏晶科技（STC）推出的基于 MCS-51 内核的 STC89C52 微控制器进行设计。

STC89C52 是一款经典的 8 位低功耗微控制器，在本电子时钟系统中，STC89C52 负责通过定时器中断实现精准计时、执行按键扫描与消抖算法，并通过其 I/O 接口与 LCD1602 显示屏、独立按键模块以及语音播报电路灵活连接，以确保系统的稳定运行与良好的人机交互体验。

3.2 LCD1602 液晶显示屏

LCD1602 字符型液晶显示模块是一款在工业控制与嵌入式领域广泛应用的经典显示组件，内部集成了 HD44780 兼容控制器及字符发生器（CGROM），能够同时显示 2 行、每行 16 个共 32 个 ASCII 码字符。该模块采用标准的并行接口设计，通过 RS 寄存器选择、R/W 读写信号及 E 使能端进行精准控制，并支持 8 位和 4 位两种数据传输模式，大大节省了单片机的 I/O 口资源。另外具备低功耗、高可靠性及对比度可调等特点，应用于各类电子系统的人机交互界面中。

本系统依靠 LCD1602 显示屏来实时呈现电子时钟的时间、日期及菜单设置状态，通过 GPIO 引脚与 STC89C52 主控芯片进行通信，随后主控芯片将处理后的时间与闹钟数据转换为标准字符指令发送至屏幕，实现直观、清晰的可视化信息反馈与人机交互功能。其模块 Proteus 仿真图（3-1）如下：

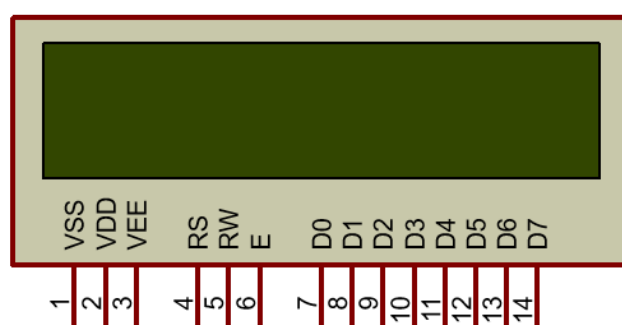


图 3-1 LCD1602 液晶显示屏仿真图

3.3 七路独立按键

七路独立按键模块是本系统核心的人机交互输入组件，由七个独立的机械轻触开关组成，每个按键单独占用一根单片机的 I/O 口线。其电路结构通常采用一端接地、另一端接控制引脚的设计，利用单片机内部的上拉电阻使引脚在常态下保持高电平，当按键按下时引脚被拉低至低电平，从而触发相应的逻辑判断。这种独立式按键结构配置灵活，软件编程简单，能够确保各个功能键之间互不干扰，是实现系统参数设置与模式切换的基础硬件保障。

本系统依靠这 7 路独立按键来实时接收用户的操作指令，通过轮询或外部中断的方式与 STC89C52 主控芯片进行通信。在实际应用中，程序会对采集到的按键信号进行软件消抖处理，随后主控芯片根据按下的具体键值执行相应的功能子

程序，包括电子时钟的时间校准、闹钟时间的设定、主菜单与调时界面的切换以及蜂鸣器提示音的开关控制等。其模块 Proteus 仿真图（3-2）如下：

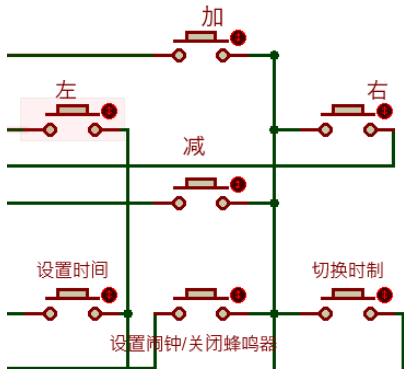


图 3-2 七路独立按键仿真图

3.4 有源蜂鸣器

有源蜂鸣器是一种在嵌入式系统中广泛应用的集成化电声报警器件，其内部自带了多谐振荡器（即内置振荡源）和驱动电路。与无源蜂鸣器不同，它无需外部提供特定频率的方波信号，只需施加额定的直流电压即可触发内部电路工作，从而发出固定频率且清晰响亮的提示音。这种“自激式”的设计使其具备驱动简单、可靠性高、响应速度快等显著优势，非常适合对音调变化要求不高但需要稳定发声的场合。

本系统依靠有源蜂鸣器来实现闹钟提醒、整点报时，直接由 I/O 口电平控制与 STC89C52 主控芯片进行连接。当系统检测到设定的闹钟时间到达或用户按下特定功能键时，主控芯片只需输出简单的高/低电平信号即可直接驱动蜂鸣器发声，极大地简化了软件编程的复杂度，并确保了在各种运行状态下声音提示的稳定性和实时性。其模块 Proteus 仿真图（3-3）如下：

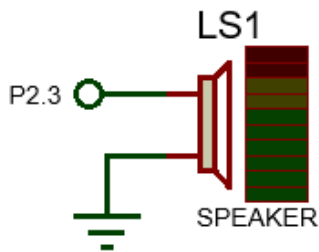


图 3-3 有源蜂鸣器仿真图

3.5 语音报时模块

由于 Proteus 仿真环境中缺乏现成的实体语音芯片模型，本系统的语音报时功能采用“虚拟串口桥接+上位机模拟”的创新方案实现。该方案通过软件协同工作构建了一条完整的仿真通信链路：首先利用 VSPD（Virtual Serial Port Driver）虚拟串口软件（如图 3-4 所示）在计算机内部建立一对相互映射的虚拟串口（例如 COM14 与 COM15），使其形成透明的数据传输通道；随后将 Proteus 中的 COMPIM 串口模块（如图 3-5 所示）配置为 9600 波特率并绑定至 COM14 端口，同时开启使用易语言自主编写的模拟串口语音上位机软件（如图 3-6 所示）并监听 COM15 端口。

系统依靠 STC89C52 主控芯片的 UART 串口发送整点报时的文本指令数据，数据经由 Proteus 的 COMPIM 模块传入 COM14，并通过 VSPD 建立的虚拟链路实时透传至 COM15。上位机软件接收到来自单片机的串口指令后，调用 Windows 系统的 TTS（Text-to-Speech）语音合成引擎或预设的音频文件进行解析与播报，从而在纯软件仿真的环境下完美复现了硬件语音模块的交互逻辑与实际效果。

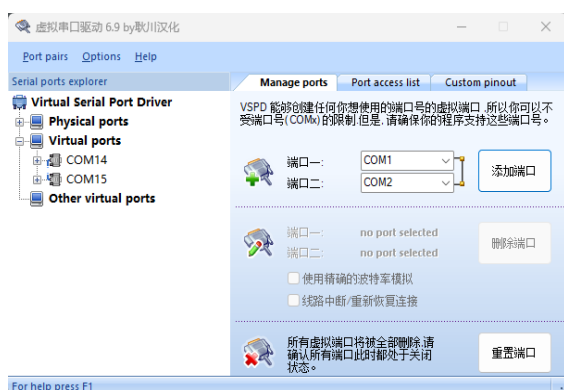


图 3-4 VSPD 虚拟串口软件



图 3-6 模拟串口语音上位机

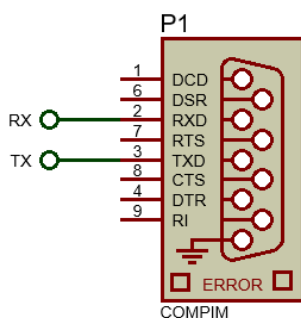


图 3-5 COMPIM 串口模块仿真图

3.6 Proteus 仿真设计

基于以上系统硬件设计简述，借助 Proteus 仿真设计绘制系统连接图，系统连接主要包含如下图几个部分：（图 3-6）

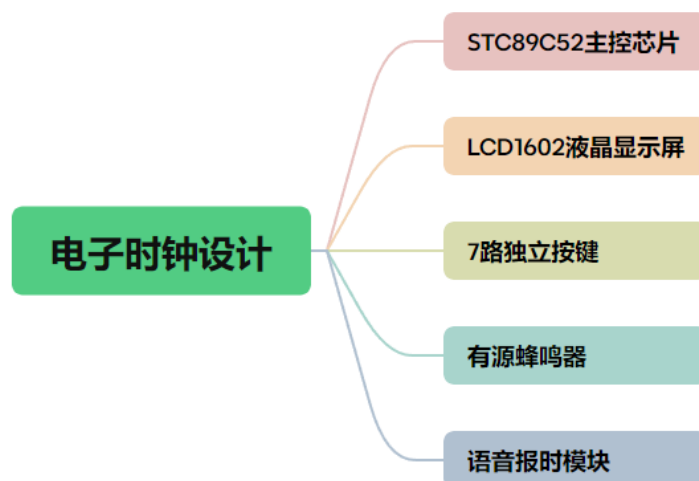


图 3-6 系统连接图组成

Proteus 仿真设计绘制效果图如下：（图 3-7）

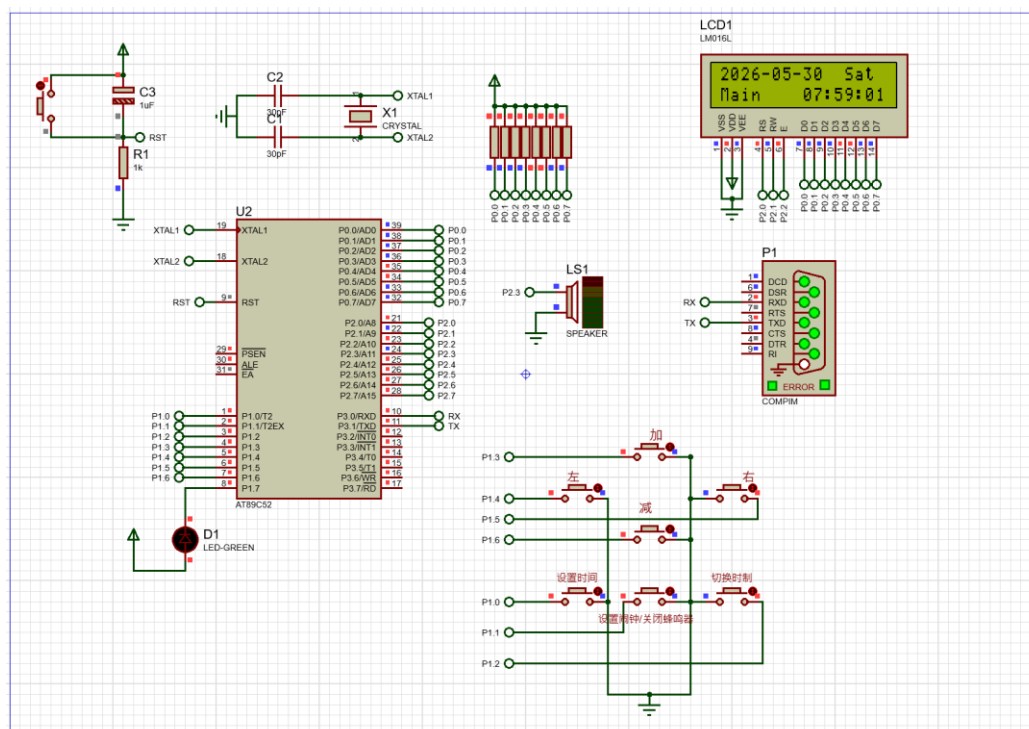


图 3-7 Proteus 仿真设计绘制效果图

引脚；而在追求速度时则采用 8 位模式一次传输完整字节，显著提升了底层驱动的适配性与开发效率，关键实现代码如下：

```
/* 写命令到 LCD。*/  
void LCD_WriteCommand(unsigned char Command)  
{  
    LCD_RS=0;  
    LCD_RW=0;  
    LCD_DataPort=Command;  
    LCD_EN=1;  
    LCD_Delay();  
    LCD_EN=0;  
    LCD_Delay();  
}  
/* 写数据到 LCD（显示字符）。*/  
void LCD_WriteData(unsigned char Data)  
{  
    LCD_RS=1;  
    LCD_RW=0;  
    LCD_DataPort=Data;  
    LCD_EN=1;  
    LCD_Delay();  
    LCD_EN=0;  
    LCD_Delay();  
}
```

4.3 七路独立按键

七路独立按键模块是本系统核心的人机交互输入组件，由七个独立的机械轻触开关组成。其电路结构通常采用一端接地、另一端接单片机的 I/O 口的设计，利用单片机内部的上拉电阻使引脚在常态下保持高电平，当按键按下时引脚被直接拉低至低电平，从而触发相应的逻辑判断。这种独立式按键结构配置灵活，能够确保各个功能键之间互不干扰，是实现系统参数设置与模式切换的基础硬件保障。

本系统依靠这七路独立按键来实时接收用户的操作指令，通过轮询的方式在主程序中持续扫描 P1 口的电平状态。在实际应用中，程序会对采集到的按键信号进行软件消抖处理以确保指令的准确性，随后主控芯片根据按下的具体键值执行相应的功能子程序。关键实现代码如下：

```
unsigned char GetKey(void)  
{  
    unsigned char KeyNumber=0;  
    if(P1_0==0){Delay(20);while(P1_0==0);Delay(20);KeyNumber=1;}  
    if(P1_1==0){Delay(20);while(P1_1==0);Delay(20);KeyNumber=2;}  
    if(P1_2==0){Delay(20);while(P1_2==0);Delay(20);KeyNumber=3;}  
    if(P1_3==0){Delay(20);while(P1_3==0);Delay(20);KeyNumber=4;}  
    if(P1_4==0){Delay(20);while(P1_4==0);Delay(20);KeyNumber=5;}  
    if(P1_5==0){Delay(20);while(P1_5==0);Delay(20);KeyNumber=6;}  
    if(P1_6==0){Delay(20);while(P1_6==0);Delay(20);KeyNumber=7;}  
    // if(P1_7==0){Delay(20);while(P1_7==0);Delay(20);KeyNumber=8;}  
    return KeyNumber;  
}
```

4.4 有源蜂鸣器

有源蜂鸣器是一种内部集成振荡源的发声器件，只需提供直流工作电压即可发出固定频率的声响，区别于需要外部方波激励的无源蜂鸣器。其典型工作电压为 3.3V~5V，驱动电流约 20~30mA，可使用单片机 I/O 口直接驱动或通过三极管放大驱动。本系统选用有源蜂鸣器，简化了软件控制逻辑——I/O 口输出高电平则鸣响，输出低电平则静音，非常适合用于闹钟提醒、按键反馈等开关量报警场景。

本系统直接利用 STC89C52 的标准弱上拉模式下的低电平灌电流驱动，实际测试工作正常，软件实现层面，封装了一个 `buzzer(int state)` 函数，通过传入非零值（如 1）使 P2.3 输出高电平启动蜂鸣器，传入 0 则输出低电平关闭。该函数在闹钟模块 `AlarmState()` 中被调用：当系统时间与闹钟设定值匹配且闹钟开关打开时，周期性或持续调用 `buzzer(1)` 发出报警声；用户按下“菜单/返回”键后调用 `buzzer(0)` 消音。关键实现代码如下：

```
//闹钟状态
void AlarmState(void)
{
    if (AlarmOnOff) {
        // 判断闹钟开关是否开启（1 为开启）
        // 判断当前时间（时、分）是否与设定的闹钟时间完全匹配
        if (RealHour == AlarmHour && TimeMinute == AlarmMinute) {
            AlarmTriggered = 1; // 将闹钟触发标志置为 1，表示闹钟正在响
            buzzer(1); // 调用蜂鸣器函数并传入 1，开启蜂鸣器报警
        }
    } else {
        // 如果闹钟开关处于关闭状态（0）
        buzzer(0); // 强制关闭蜂鸣器，确保闹钟不会误响
    }
}

// 蜂鸣器控制函数
void buzzer(int state)
{
    if (state) P2_3 = 1; // 如果传入的 state 为非 0（真），则将 P2.3 引脚置高电平，驱动蜂鸣器响
    else P2_3 = 0; // 如果传入的 state 为 0（假），则将 P2.3 引脚置低电平，关闭蜂鸣器
}
```

4.5 语音报时模块

本系统采用串口通信方式控制外接的语音播报模块，实现整点报时、语音模块通过串口接收指令帧，解析后合成语音输出。因此，需对 STC89C52 单片机的串口进行初始化配置。

4.5.1 串口通信初始化

串口通信初始化是语音报时功能的基础，主要完成波特率设置、数据帧格式定义及中断配置。本系统选用定时器 T1 作为波特率发生器，工作于 **8 位自动重装模式**，配合 11.0592MHz 晶振产生 9600bps 的通信速率该速率是语音模块常用的标准速率，误码率低且传输可靠。

数据发送函数 `Uartsend(unsigned char byte)` 将待发送字节写入 SBUF 寄存器，然后等待硬件置位 TI 标志（表示发送完成），最后手动清零 TI 以便下一次发送。中断服务函数 `UART_Routine(void) __interrupt(4)` 仅对接收中断 RI 进行清零处理，关键实现代码如下：

```
//串口初始化

void InitUART(void) //9600bps@11.0592MHz
{
    PCON &= 0x7F; //波特率不倍速
    SCON = 0x40; //8 位数据,可变波特率
    TMOD &= 0x0F; //清除定时器 1 模式位
    TMOD |= 0x20; //设定定时器 1 为 8 位自动重装方式
    TL1 = 0xFD; //设定定时初值
    TH1 = 0xFD; //设定定时器重装值
    TR1 = 1; //启动定时器 1
    ET1 = 0; //禁止定时器 1 中断
}

void Uartsend(unsigned char byte) //发送
{
    SBUF = byte; //把数据写入发送缓冲区 SBUF
    //数据发送完成的标志是 TI=1; 所以等待数据传送完
    while (TI == 0);
    TI = 0; //软件清零
}

void UART_Routine(void) __interrupt(4)
{
    if (RI) {
        RI = 0;
    } else
        TI = 0;
}
```

4.5.2 串口通信协议

语音报时功能需要将时间信息封装为特定格式的指令帧，通过串口发送给外接语音合成模块。本系统定义了一套基于字节流的简易通信协议：每个报时指令帧由起始标识符 [、报时内容文本（GBK 编码的中文字符串与数字字符）以及结束标识符] 三部分组成，模块收到完整帧后立即合成语音并播报。

表 4-1 串口通信协议

字段	长度	说明
起始符	1	[（ASCII 0x5B），标识一帧开始
报时内容	可变	GBK 编码的中文文本 + ASCII 数字字符
结束符	1	]（ASCII 0x5D），标识一帧结束

关键实现代码如下：

```
// 整点报时调度函数
void TelTime(unsigned int Hour_flag, unsigned int hour,
             unsigned int minute, unsigned int sec)
{
    if (minute == 0 && sec == 0 && (hour >= 8 && hour <= 22)) {
        if (Hour_flag == 0) {
            SendBeijingTime(hour); // 24 小时制播报
        } else {
            SendBeijingTime_12H(hour); // 12 小时制播报
        }
    }
}

// 发送 24 小时制报时帧
void SendBeijingTime(unsigned int hour)
{
    Uartsend('[');
    // “现在是北京时间” GBK 码（14 字节）
    Uartsend(0xCF); Uartsend(0xD6); // 现
    Uartsend(0xD4); Uartsend(0xDA); // 在
    Uartsend(0xCA); Uartsend(0xC7); // 是
    Uartsend(0xB1); Uartsend(0xB1); // 北
    Uartsend(0xBE); Uartsend(0xA9); // 京
    Uartsend(0xCA); Uartsend(0xB1); // 时
    Uartsend(0xBC); Uartsend(0xE4); // 间
    // 发送小时数字（两位，不足十位补前导零）
    if (hour >= 10) {
        Uartsend('0' + hour / 10);
        Uartsend('0' + hour % 10);
    } else {
        Uartsend('0' + hour);
    }
    // “点整” GBK 码
    Uartsend(0xB5); Uartsend(0xE3); // 点
    Uartsend(0xD5); Uartsend(0xFB); // 整
    Uartsend(']');
}
```

第五章 核心算法

5.1 时间核心逻辑

时间核心逻辑是整个电子时钟系统的数据中枢，负责维护秒、分、时、日、月、年六个时间维度的准确性与连续性。所有与时间相关的功能——包括 LCD 主界面显示、闹钟比对、整点报时触发、星期推算等——均依赖于一套统一且可靠的时间变量（TimeSec、TimeMinute、RealHour、TimeDay、TimeMonth、TimeYear）。

5.1.1 时间核心逻辑简述

Calendar() 函数采用自底向上、逐级进位的方式维护时间：秒满 60 向分进位，分满 60 向时进位，时满 24 向日进位。日进位时调用 GetDaysInMonth() 获取当月实际天数（自动处理闰年及大小月），跨月或跨年时相应调整月份和年份。所有操作均为轻量级比较与赋值，无阻塞延时。

该函数与定时器中断协同工作：中断每秒对 TimeSec 加 1，Calendar() 仅在各边界时刻执行进位，既保证计时精度，又避免中断内复杂运算。关键实现代码如下：

```
void Calendar(void)
{
    // 秒进位逻辑
    if (TimeSec >= 60) {
        // 如果当前秒数累计达到或超过 60 秒
        TimeSec = 0; // 秒数清零，重新开始计数
        TimeMinute++; // 分钟数加 1（进位）
    }
    // 分进位逻辑
    if (TimeMinute >= 60) {
        // 如果当前分钟数累计达到或超过 60 分钟
        TimeMinute = 0; // 分钟数清零
        RealHour++; // 底层 24 小时制的小时数加 1（进位）
    }
    // 时进位逻辑（底层逻辑永远是 0-23）
    if (RealHour >= 24) {
        // 如果小时数累计达到或超过 24 小时
        RealHour = 0; // 小时数清零（新的一天开始）
        TimeDay++; // 日期（天数）加 1（进位）
    }
    unsigned char maxDay = GetDaysInMonth(TimeYear, TimeMonth);
    if (TimeDay > maxDay) {
        // 如果当前日期超过了该月的最大天数
        TimeDay = 1; // 日期重置为 1 号
        TimeMonth++; // 月份加 1（进位到下一个月）
        if (TimeMonth > 12) {
            // 如果月份超过了 12 月
            TimeMonth = 1; // 月份重置为 1 月
            TimeYear++; // 年份加 1（进入新的一年）
        }
    }
}
```

5.1.2 计算星期核心逻辑

星期推算采用基姆拉尔森计算公式，输入年、月、日即可返回对应的星期（0=星期日，1=星期一……6=星期六）。该公式的核心处理在于：将每年的一月、二月视为上一年的十三月、十四月，以便统一计算。随后提取年份后两位 y 和世纪数 c ，代入公式 $(\text{day} + 26 * (\text{month} + 1) / 10 + y + y / 4 + c / 4 + 5 * c) \% 7$ 得到原始结果，再通过 $(w + 6) \% 7$ 将值域调整至 0~6（星期日为 0）。该算法无需查表，计算量小，适合嵌入式环境。关键实现代码如下：

```
// 计算星期几（基姆拉尔森公式）
// 返回值: 0=星期日, 1=星期一, ..., 6=星期六
unsigned char GetWeekday(unsigned int year, unsigned char month, unsigned char day)
{
    // 将 1 月、2 月看作上一年的 13 月、14 月
    if (month == 1 || month == 2) {
        month += 12;
        year--;
    }
    unsigned int y = year % 100; // 年份后两位
    unsigned int c = year / 100; // 年份前两位（世纪数）
    // 基姆拉尔森公式（系数 26*(m+1)/10 等价于 13*(m+1)/5）
    unsigned int w = (day + (26 * (month + 1)) / 10 + y + y / 4 + c / 4 + 5 * c) % 7;

    w = (w + 6) % 7;
    return (unsigned char) w;
}
```

5.1.2 按键功能核心逻辑

按键处理是整个系统人机交互的枢纽。KeyLogic() 函数通过 switch-case 结构对 7 个独立按键的键值进行分支响应，并根据当前界面标志 Menu_flag（0=主界面，1=时间设置，2=闹钟设置）执行不同操作

```
// --- 按键逻辑 ---
void KeyLogic(int key) {
    switch (key) {
        case 1: // 确认键
            if (Menu_flag == 0) { LCD_Clear(); SwopAlter(); Menu_flag = 1; }
            else { LCD_Clear(); AlterSwop(); Menu_flag = 0; }
            break;
        case 2: // 菜单/返回键
            if (AlarmTriggered == 0) {
                if (Menu_flag == 0) { LCD_Clear(); Menu_flag = 2; }
                else { LCD_Clear(); Menu_flag = 0; }
            } else if (AlarmTriggered == 1) { AlarmTriggered = 0; buzzer(0); }
            break;
        case 3: // 时制切换
            TimeHour_flag = !TimeHour_flag; LCD_Clear();
            break;
        case 4: // 加键
            if (Menu_flag == 1) {
                switch (AlterFlag) {
                    case 1: if (++AlterSec > 59) AlterSec = 0; break;
                    case 2: if (++AlterMinute > 59) AlterMinute = 0; break;
                    case 3: if (++AlterRealHour > 23) AlterRealHour = 0; break;
                    case 4: if (++AlterDay > GetDaysInMonth(AlterYear, AlterMonth)) AlterDay = 1; break;
                    case 5: if (++AlterMonth > 12) AlterMonth = 1; { unsigned char maxDay = GetDaysInMonth(AlterYear, AlterMonth); if (AlterDay > maxDay) AlterDay = maxDay; } break;
                    case 6: if (++AlterYear > 9999) AlterYear = 1900; break;
                }
            } else if (Menu_flag == 2) {
                switch (AlarmFlag) {
                    case 2: if (++AlarmMinute > 59) AlarmMinute = 0; break;
                    case 3: if (++AlarmHour > 23) AlarmHour = 0; break;
                    case 1: AlarmOnOff = !AlarmOnOff; break;
                }
            }
    }
}
```

```

        break;

    case 7: // 减键
        if (Menu_flag == 1) {
            switch (AlterFlag) {
                case 1: if (AlterSec-- == 0) AlterSec = 59; break;
                case 2: if (AlterMinute-- == 0) AlterMinute = 59; break;
                case 3: if (AlterRealHour-- == 0) AlterRealHour = 23; break;
                case 4: if (AlterDay-- == 1) AlterDay = GetDaysInMonth(AlterYear, AlterMonth); break;
                case 5: if (AlterMonth == 1) AlterMonth = 12; else AlterMonth--; { unsigned char maxDay =
GetDaysInMonth(AlterYear, AlterMonth); if (AlterDay > maxDay) AlterDay = maxDay; } break;
                case 6: if (AlterYear-- == 1900) AlterYear = 9999; break;
            }
        } else if (Menu_flag == 2) {
            switch (AlarmFlag) {
                case 2: if (AlarmMinute == 0) AlarmMinute = 59; else AlarmMinute--; break;
                case 3: if (AlarmHour == 0) AlarmHour = 23; else AlarmHour--; break;
                case 1: AlarmOnOff = !AlarmOnOff; break;
            }
        }
        break;

    case 5: // 右移
        if (Menu_flag == 1) { if (++AlterFlag > 6) AlterFlag = 1; }
        else if (Menu_flag == 2) { if (++AlarmFlag > 3) AlarmFlag = 1; }
        break;

    case 6: // 左移
        if (Menu_flag == 1) { if (--AlterFlag < 1) AlterFlag = 6; }
        else if (Menu_flag == 2) { if (--AlarmFlag < 1) AlarmFlag = 3; }
        break;
    }
    KeyValue = 0;
}

```

第六章 调试与实现

6.1 软件调试与分析

① 调试按键时发现加按键能正常工作，减按键无法正常工作。在 KeyLogic() 函数中设置断点，分别按下加键（Key=4）和减键（Key=7）进行单步跟踪。发现加键分支正常执行变量自增操作，而减键分支执行后变量值未发生变化。进一步检查代码，发现减键对应的 case 7 分支末尾 忘记写 break，导致程序在执行完减键逻辑后继续向下执行其他 case 分支，最终变量值被覆盖。在 case 7 末尾添加 break 语句后，减按键恢复正常工作。

② 多个功能共用同一按键时出现功能冲突例如“确认键”在主界面用于进入设置，在设置界面用于保存返回，但调试时发现在闹钟界面按确认键也会触发时间设置界面。检查 KeyLogic() 中的 case 1 分支，发现仅判断了 Menu_flag==0 和 else，未排除 Menu_flag==2 的情况。修正为使用 if-else if-else 结构，分别处理三种界面状态，功能冲突解决。

6.2 proteus 仿真操作方法

本系统通过 7 个独立按键 实现人机交互，各按键功能定义如下：

表 6-2 按键功能表

按键编号	名称	说明
键 1	时间设置	主界面下进入时间设置界面；设置界面下保存修改并返回主界面
键 2	闹钟设置	主界面下进入闹钟设置界面；其他界面下返回主界面；闹钟响时关闭闹钟
键 3	时制切换	切换 12/24 小时制显示（不影响底层计时）
键 4	数值增加	数值增加
键 5	数值右移	光标向右移动（切换设置字段）
键 6	数值左移	光标向左移动
键 7	数值减少	数值减少

6.2.1 主界面操作方法

- ① 上电后默认进入 主界面，显示当前时间、日期、星期、AM/PM。
- ② 按 键 1：进入时间设置界面。
- ③ 按 键 2：进入闹钟设置界面。
- ④ 按 键 3：切换 12/24 小时制。
- ⑤ 按 加/减键：无作用（仅在设置界面有效）。
- ⑥ 闹钟到达设定时间时，蜂鸣器鸣响，按 键 2 可关闭闹钟。

6.2.2 时间设置操作方法

- ① 按 键 5/键 6：左右移动光标，可选择 秒→分→时→日→月→年 字段。
- ② 按 键 4/键 7：对光标所在字段进行 加/减 操作，数值自动进位或回绕。
- ③ 按 键 1：保存所有修改并返回主界面。
- ④ 按 键 2：不保存修改直接返回主界面。

6.2.3 时间设置操作方法

- ① 按 键 5/键 6：左右移动光标，可选择 小时 → 分钟 → 开关 字段。
- ② 按 键 4/键 7：对光标所在字段进行 加/减 或 ON/OFF 切换。
- ③ 按 键 1：返回主界面（闹钟参数已实时保存）。
- ④ 按 键 2：返回主界面。

第七章 总结

本文基于 STC89C52 单片机设计并实现了一款电子时钟系统。系统硬件包括 LCD1602 液晶显示屏、七路独立按键、有源蜂鸣器及串口语音播报模块；软件采用模块化编程，实现了时间日期显示、闹钟提醒、整点报时、12/24 小时制切换、时间日期设置、闹钟设置等核心功能。

系统以定时器中断提供 1ms 时基，通过 Calendar() 函数完成秒、分、时、日、月、年的分层进位与闰年修正。星期计算采用基姆拉尔森公式，无需查表。按键逻辑基于状态机管理主界面、时间设置界面、闹钟设置界面三个状态，支持光标移动、数值增减、边界回绕及日期合法性自动修正。闹钟到达设定时间时触发蜂鸣器，按下菜单键可关闭；整点报时在 8:00 至 22:00 之间通过串口发送语音指令，支持 12/24 小时制播报格式。

在 Proteus 仿真环境下完成了功能调试，解决了按键消抖、闰年判断错误、闹钟标志位逻辑缺陷、12 小时制边界显示等问题，系统运行稳定。

第八章 参考文献

- [1] 李朝青. 单片机原理及接口技术（第 5 版）[M]. 北京：北京航空航天大学出版社，2017.
- [2] 郭天祥. 新概念 51 单片机 C 语言教程——入门、提高、开发、拓展全攻略[M]. 北京：电子工业出版社，2018.
- [3] 谭浩强. C 程序设计（第五版）[M]. 北京：清华大学出版社，2017.
- [4] 张毅刚. 单片机原理及应用（第三版）[M]. 北京：高等教育出版社，2016.
- [5] 周润景，张丽娜. 基于 Proteus 的单片机系统设计与仿真[M]. 北京：电子工业出版社，2019.
- [6] 何立民. 单片机高级教程：应用与设计（第 2 版）[M]. 北京：北京航空航天大学出版社，2015.
- [7] 刘火良，杨森. STC15 单片机实战指南（从入门到精通）[M]. 北京：机械工业出版社，2017.
- [8] 赵亮，侯国锐. 单片机 C 语言编程与 Proteus 仿真[M]. 北京：人民邮电出版社，2012.
- [9] 沈红卫. 基于单片机的智能控制技术的应用[M]. 北京：电子工业出版社，2014.
- [10] 王建校，张虹. 单片机应用系统设计与仿真调试[M]. 西安：西安电子科技大学出版社，2018.

第九章 附录

为了让更多对电子时钟系统感兴趣的人能够了解、学习并从中受益，本项目的完整代码、硬件电路设计及相关文档将开源并发布在我的 GitHub 平台：

[Nose-Alien/C51-Electronic-clock-design](https://github.com/Nose-Alien/C51-Electronic-clock-design)

通过开源项目，希望为学习嵌入式开发、单片机应用和人机交互技术的爱好者提供一个实践案例，同时也为相关领域的研究者和开发者提供参考与灵感。欢迎通过评论或提交 Issue 的方式进行交流，推动共同学习与进步，探索更多可能性。